

# IMPLEMENT ERROR DETECTION AND CORRECTION USING THE HAMMING CODE TECHNIQUE

## AIM

To **implement error detection and correction using the Hamming Code technique** at the Data Link Layer, by converting the input text into binary, adding redundant bits, detecting and correcting single-bit errors, and recovering the original data at the receiver.

## Sender and Receiver Algorithm

### Sender Algorithm

1. Read original data.
2. Convert data into binary.
3. Calculate number of parity bits:

$$2^r \geq m + r + 1$$

4. Create positions 1 to n.
5. Place parity bits at:  
  
1, 2, 4, 8, ...
6. Place data bits in remaining positions.
7. Calculate each parity bit using even parity.
8. Form the Hamming code.
9. Send/store the encoded data in the channel.

### Receiver Algorithm

1. Read Hamming code from the channel.
2. Check every parity group.
3. Calculate P1, P2, P4, P8, ...
4. Combine the parity-check results.
5. Obtain the syndrome.

6. If syndrome = 0:

No error.

7. If syndrome != 0:

Syndrome gives error position.

8. Flip the bit at that position.

9. Remove parity bits.

10. Recover the original data.

### Program:

```
#include <stdio.h>

void encode(int data[4], int code[7])
{
    int p1, p2, p4;

    code[2] = data[0];
    code[4] = data[1];
    code[5] = data[2];
    code[6] = data[3];
    p1 = code[2] ^ code[4] ^ code[6];
    p2 = code[2] ^ code[5] ^ code[6];
    p4 = code[4] ^ code[5] ^ code[6];

    code[0] = p1; // Position 1
    code[1] = p2; // Position 2
    code[3] = p4; // Position 4
}

void decode(int code[7])
{
    int p1, p2, p4;
    int errorPosition;

    p1 = code[0] ^ code[2] ^ code[4] ^ code[6];
    p2 = code[1] ^ code[2] ^ code[5] ^ code[6];
    p4 = code[3] ^ code[4] ^ code[5] ^ code[6];

    errorPosition = p4 * 4 + p2 * 2 + p1;

    if (errorPosition == 0)
    {
        printf("No error detected.\n");
    }
    else
    {
        printf("Error detected at position: %d\n", errorPosition);
    }
}
```

```

        code[errorPosition - 1] ^= 1;

        printf("Error corrected successfully.\n");
    }

    printf("Corrected Hamming Code: ");

    for (int i = 0; i < 7; i++)
        printf("%d", code[i]);

    printf("\n");

    printf("Original Data: %d%d%d%d\n",
           code[2], code[4], code[5], code[6]);
}

int main()
{
    int data[4];
    int code[7];

    printf("Enter 4-bit data: ");

    for (int i = 0; i < 4; i++)
        scanf("%d", &data[i]);

    encode(data, code);

    printf("\nEncoded Hamming Code: ");

    for (int i = 0; i < 7; i++)
        printf("%d", code[i]);

    printf("\n");

    code[5] ^= 1;

    printf("Received Code with Error: ");

    for (int i = 0; i < 7; i++)
        printf("%d", code[i]);

    printf("\n");

    decode(code);

    return 0;
}

```

## Result:

Thus, error detection and correction using **Hamming Code at the Data Link Layer** was successfully implemented and verified.